
ENGGEN 499

Practical Work Experience Report

Pratham Chhabra

Department of Electrical, Computer, and Software Engineering

Monday 23 March, 2025



Waipapa
Taumata Rau
**University
of Auckland**

Summer Research Assistant
Work Period: 24/11/2025 to 20/06/2025

ACKNOWLEDGEMENTS

I would like to express my appreciation towards my supervisor, Dr Morteza Biglari-Abhari, for their unmatched support for this project, who gave me an ample amount of advice and feedback throughout the few months I was there.

I would also like to thank the technicians in the Embedded Systems Lab, Elaine and Sunitha, who answered any questions I had about which tools and software I could access and provided me with access to the laboratory at the beginning and end of my project, when I had lost access to the room.

SUMMARY

This report details my tenure as a Summer Research Assistant at the University of Auckland within the Department of Electrical, Computer, and Software Engineering. My work focused on the development and hardware verification of a reconfigurable Neural Processing Unit (NPU) prototyped on an Intel Arria 10 FPGA. The primary objective was to architect a system capable of dynamically adjusting its computational precision and exploiting structured sparsity to improve inference throughput. Throughout the project, I was responsible for the full design lifecycle, which included RTL development in VHDL, timing closure in Quartus Pro, and the creation of a deployment pipeline using the OpenVINO toolkit. By implementing a dual-core architecture, I successfully demonstrated significant latency reductions across several vision models, including between 30-40% gains in efficiency. This role provided me with significant insight into professional research and academia, requiring high levels of technical knowledge and strategic project management.

Contents

Acknowledgements	2
1 Introduction	6
2 Workplace Information	6
2.1 Workspace Layout/Facilities	6
2.2 Personal Setup	7
3 Work Experience	7
3.1 Prior Work	7
3.2 Phase 1: New Research	8
3.3 Phase 2: Attempting Dynamic Partial Reconfiguration	9
3.4 Phase 3: Multi-Core NPU	12
3.4.1 Optimisations Performed	15
3.4.2 Results	16
3.5 Reflective Appraisal	18
3.6 Conclusion	19

List of Figures

1	Wide Angle View of the University of Auckland Faculty of Engineering Building . . .	6
2	Summer Research Setup From Angled View	7
3	Summer Research Setup from Closeup View	7
4	Close-up of the Arria 10 FPGA Development Board	8
5	Proposed System Diagram of a Dynamic Partial Reconfiguration-Based NPU	9
6	Theoretical Timing Diagram Explaining the DPR operation	10
7	Test Simulations of my Ping Pong Buffer	10
8	Theoretical Maximum Efficiency Achieved for each Tile and precision flexibility	11
9	Theoretical Average efficiency achieved for each Quantisation precision	11
10	Complete System Diagram	14
11	AlexNet Performance: 36.25% Total Latency Reduction.	16
12	MobileNetV2 Performance: 34.50% Total Latency Reduction.	16
13	ResNet50 Performance: 38.16% Total Latency Reduction.	17
14	VGG16 Performance: 32.37% Total Latency Reduction.	17

1 INTRODUCTION

From November 2025 to March 2026, I had the privilege of working as a Summer Research Assistant under the supervision of the Department of Electrical, Computer, and Software Engineering at the University of Auckland. The research was conducted on Level 7 of the 405 building within the Embedded Systems Laboratory. The project I worked on involved developing a reconfigurable NPU architecture to accelerate Convolutional Neural Networks (CNNs). My specific focus was to build upon my previous part IV simulation work and transform it into a hardware-verified prototype on the Arria 10 platform. This involved a significant amount of independent research into the FPGA's capabilities and other toolkits I may need to build an inference pipeline. At the same time, I investigated various architectural approaches, such as Dynamic Partial Reconfiguration and Soft-core processor integration, for additional management.

Throughout this experience, I have operated as a proactive research engineer, responsible for architecting the main hardware, managing software-to-hardware handoffs, and documenting results for future departmental use. This role has not only broadened my technical expertise in FPGA timing closures, TCL automations, and Python-based model optimisations but has also given me firsthand experience in the investigative and iterative nature of professional R&D.

2 WORKPLACE INFORMATION



Figure 1: Wide Angle View of the University of Auckland Faculty of Engineering Building

2.1 Workspace Layout/Facilities

The Faculty of Engineering and Design building at the University of Auckland, located at 20 Symonds Street, is a familiar place to me, as I completed my four years of undergraduate studies there just before taking on the role of a Research Assistant during the summer. The faculty building, being split into multiple floors based on specialisation, lecture halls, study spaces, and offices, where my research was conducted on floor 7 of the 405 building in the Embedded Systems laboratory. This laboratory contained everything I would ever need as a Computer Systems Engineer, from Robots, microcontrollers, FPGAs, and a wide variety of other computer-related devices for research-based projects.



Figure 2: Summer Research Setup From Angled View



Figure 3: Summer Research Setup from Closeup View

2.2 Personal Setup

My setup was on the opposite side of the first row of computers in the laboratory, where I used the computer adjacent to the one assigned to me as the second monitor and the one assigned for connecting to the FPGA or doing any of the technical work.

The room remained mostly empty because there were not many research students in the Embedded Systems department over the summer. So there were only the technicians in their office and two to three other summer research students working on their projects. The majority of the research students on floor 7 resided in the Control Systems and Green Technologies department.

3 WORK EXPERIENCE

Working as a Summer Research Assistant in the Embedded Systems department required a high level of technical knowledge. Unlike a standard corporate internship with daily meetings and dedicated mentors, my role mirrored a more focused R&D environment where I was responsible for my own project management. With my supervisor, available for biweekly high-level academic consultations, and sometimes weekly if things get out of hand. Because of this, I had to act as the expert for the NPU hardware and software stack. This independence necessitated a disciplined approach to self-documentation and forced me to develop a rigorous troubleshooting methodology, as there was no immediate support for the specialised hardware conflicts I encountered on the FPGA due to their complexity.

3.1 Prior Work

This project was built on a previous final-year research project that established the foundations for this NPU, which was restricted to simulations. The focus was on the fact that convolutional layers are the biggest bottleneck in vision AI, often accounting for 90% of processing time. This is why the image data are reshaped into 2D matrices. This is where a systolic array comes in to accelerate matrix multiplication. An output-stationary systolic array was established as the core engine. Within this was a grid of processing elements in which data flowed horizontally, and weights flowed vertically. In this setup, the partial sums are stationary within each PE's accumulator, simplifying VHDL logic because it would not have to route intermediate results between PEs. I calculated that the total latency for this math would be $M + N + K - 2$ clock cycles.

A coordinated row-and-column algorithm to handle structured sparsity was implemented. And by identifying these, One could simply remove them in software before they even hit the FPGA. This produced compacted tiles, allowing the hardware to skip empty calculations to finish the job much faster.

The previous iteration had several blocks that I needed to fix in this project. Firstly, there were mostly successful simulations, but not proper FPGA synthesis or inference. It was also stuck in a single 8x8 INT8 configuration, which offered no way to test different models or change them at runtime except by changing the systolic array size before compilation

3.2 Phase 1: New Research

Initial research focused on the Intel FPGA AI Suite and the OpenVINO (Open Visual Inference and Neural Network Optimisation) Toolkit. OpenVINO was identified as a critical software suite for accelerating inference by converting pre-trained models into an intermediate Representation (IR), consisting of .xml files for network structure and .bin files for weights and biases.

I explored extending my previous hardware-software co-design idea to use the HPS (Hard Processor System) and/or a Nios V processor as an additional or main control unit. This architecture would allow the processor to manage complex memory transactions and issue custom instructions to the NPU. Research into RISC-V acceleration suggested that integrating the accelerator at the Instruction Set Architecture (ISA) level could balance all three of the parameters: latency, power, and flexibility.

To overcome the limitations of on-chip memory, research on Double Buffering was conducted. Since the Arria 10's on-chip memory is insufficient for entire CNN models, data must be partitioned into smaller blocks that fit within the buffers. I am investing in a system using M20K BRAM blocks to implement a ping-pong buffer mechanism. This allows the NPU to compute a current tile while the DMA (Direct Memory Access) engine simultaneously loads the next tile into a second buffer, eliminating NPU idle time.

Simultaneously, I looked into the new board specifications as I moved from a small Cyclone V to a powerful Arria 10, shown in Figure 4. This was a massive increase in computational potential, with DSP blocks increasing from 81 to approximately 1600. The architecture of these DSP blocks enabled two 18-bit multiplications to be performed in a single block, a feature that supported my goal of dynamic precision handling and could drastically increase throughput.



Figure 4: Close-up of the Arria 10 FPGA Development Board

3.3 Phase 2: Attempting Dynamic Partial Reconfiguration

The initial phase of my research focused on mitigating the memory "blocks" common in FPGA-based accelerators, where the NPU remains idle while waiting for the next data tile to be loaded from external memory. I designed and simulated a Ping-Pong, or what is known as a double-buffering mechanism, utilising the Arria 10's M20K BRAM blocks to decouple the data-loading and computation phases. As illustrated in Figure 7, the functional simulation confirms that while the systolic array processes the current 'Active' buffer, the DMA controller can simultaneously populate the other buffer with the subsequent tile. This overlapping of communication and computation was a vital foundational step; it ensured that the system's theoretical throughput would not be entirely bottlenecked by memory transfer latencies

My most significant research undertaking involved exploring the feasibility of Dynamic Partial Reconfiguration on the Arria 10 platform. Unlike the previous project, which used static generate statements to set array dimensions at synthesis time, I wanted to achieve a system that could physically swap its core logic at runtime. This would allow the FPGA to transform from a high-precision 16-bit architecture to possibly a high-efficiency 8-bit architecture as different layers of CNN were processed.

I began by partitioning my design into two main areas, a static region and a reconfigurable region. The static region contained the backbone of my Soft-core processor, NIOS, the modular scatter-gather DMA (mSGDMA) controllers, and the Avalon-MM interconnects. I intended the reconfigurable region to be a "slot" for streaming different hardware "personas". To manage this, I utilised the logic Lock feature in Quartus to set physical boundaries on the chip floorplan. I had to be extremely precise here; I needed to ensure the swapped logic wouldn't interfere with the pre-existing high-speed routing for the DDR4 External Memory Interface (EMIF). I initially planned to use an internal host Partial Reconfiguration (PR) model, where my NIOS V processor would communicate directly with the Arria 10 PR control block via an Avalon-MM interface to stream the partial bitstream files (.rbf). Figure 5 shows my proposed design for such a system.

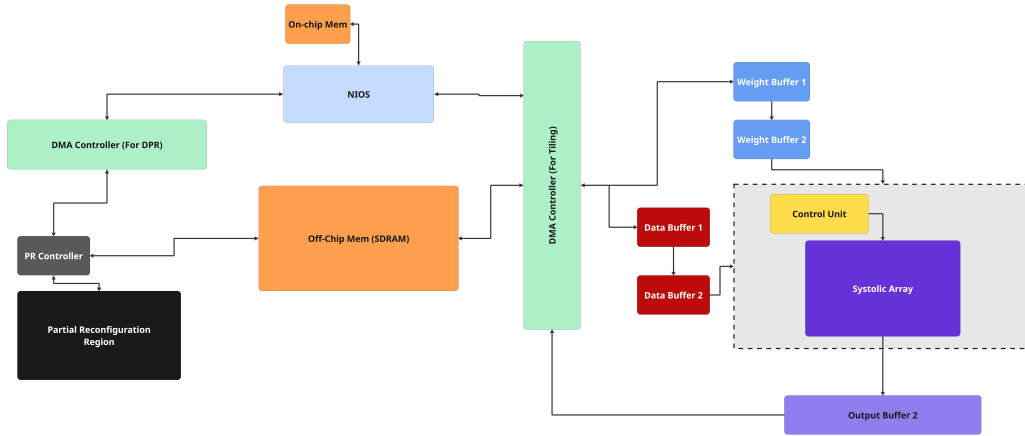


Figure 5: Proposed System Diagram of a Dynamic Partial Reconfiguration-Based NPU

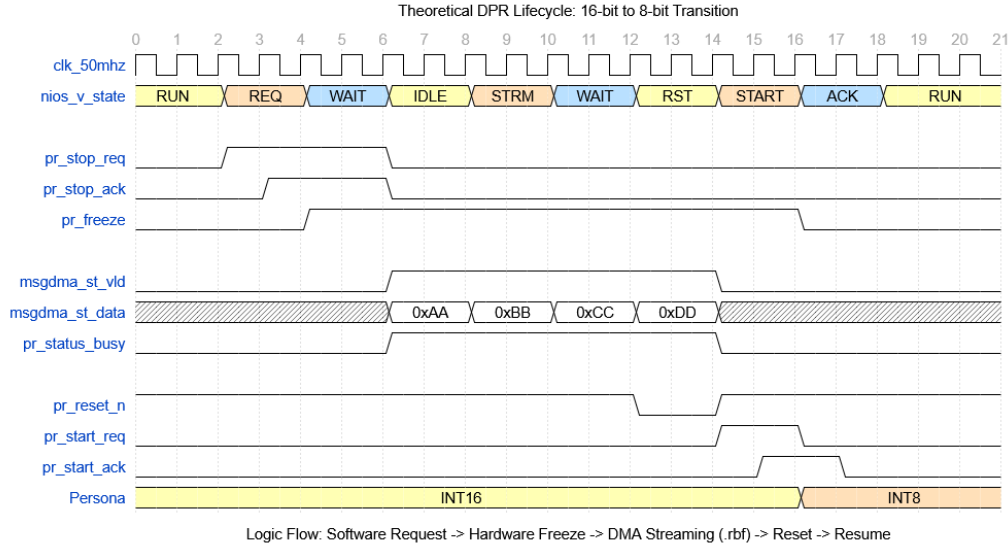


Figure 6: Theoretical Timing Diagram Explaining the DPR operation

As shown in the timing diagram in Figure 6, the DPR operation's logical flow would be managed by a Nios V soft-core processor, which would act as the system orchestrator. The process would start with a software-initiated request to switch hardware "personas". To prevent system instability, the processor would execute a "Freeze" handshake, isolating the reconfigurable partition from the static Avalon-MM interconnects. Once isolated, the mSGDMA engine is triggered to stream the partial bitstream (.rbf) from DDR4 memory directly to the Arria 10 PR Control Block. Following a successful bitstream load, the processor would issue a local hardware reset to the new logic partition and de-assert the isolation signals, allowing the NPU to resume inference with the new computational precision without a full system reboot.

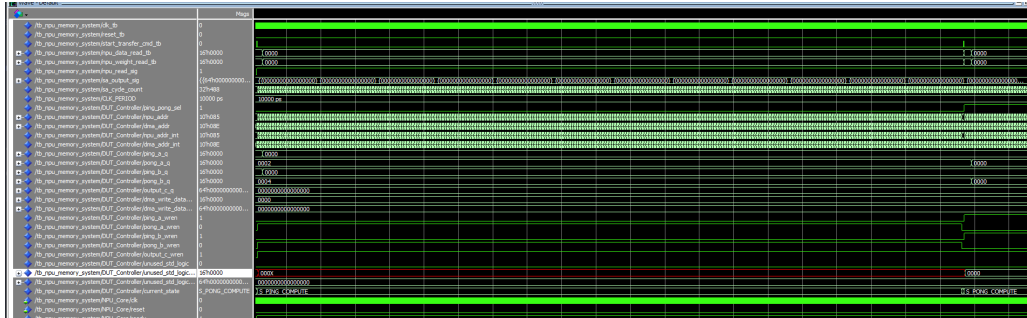


Figure 7: Test Simulations of my Ping Pong Buffer

As I moved from theory into implementation, I hit a series of critical failures that stalled my progress for weeks. The most complex issue was a Global Clock Routing deadlock. Because the PR regions and the DDR4 EMIF occupied distinct columns on the Arria 10 silicon, the Quartus fitter struggled to route the global 50 MHz clock signal.

I encountered a recurring error where the fitter would stop at 24% and complain that the PLL reference clock was constrained to an I/O bank (Bank 3G) outside the banks used by the EMIF. I spent two weeks in the pin planner performing trial-and-error assignments, reading datasheets, and attempting to resolve I/O standard conflicts between the LVDS standard required by the SDRAM and the 1.8V logic used in my system. At one point, these constraints created an infinite routing loop, causing Quartus to experience a massive memory leak. From this, the RAM usage would climb to 2925MB

before the software crashed entirely.

Even after bypassing specific routing errors, the reconfiguration data path was problematic. I attempted to use the mSGDMA to stream bitstreams to the PR controller using a Memory-Mapped-to-Streaming interface. However, I faced constant alignment errors because the DMA was outputting 4-byte symbols, while the PR controller expected a singular 32-bit symbol. The two were physically incompatible in their byte-alignment settings, forcing me to abandon the streaming approach in favour of a slower MM-to-MM transfer

After encountering significant issues with DPR, I decided to step back and perform a theoretical analysis of my target workload. I realised that to design an efficient accelerator, I first needed to demonstrate that a Convolutional Neural Network exhibited differences throughout the model flow that justified a reconfigurable approach, since there was minimal research available on this.

I developed a Python-based AlexNet performance profile using the script from the previous project had to simulate how my coordinated row-removal algorithm would perform across different physical tile sizes. The script utilised the PyTorch framework to load a pre-trained AlexNet model and applied 8-bit linear quantisation to the weights and activations. It also already had the im2col transformation and a tiling function that segmented the large convolutional matrices into 8x8 and 16x16 tiles. I then ran these tiles through my stripping algorithm to calculate the Effective Dimensions (M, N, K) and determined the theoretical cycle savings for each layer.

The data I collected from this profiling, specifically testing on complex images like a cat from the ImageNet set, revealed a fascinating pattern that directly influenced my final hardware design. I observed a U-shaped efficiency curve across the network depth. Where the outer layers (Conv1 and Conv5) showed massive speedups when using the 8x8 tile, up to 2.3x. This somewhat gave me an idea that the raw spatial input and final feature extractions could naturally be sparse, allowing the algorithm to eliminate a high percentage of cycles. The middle layers, Conv2, 3, and 4, seemed much denser. In these instances, large 32x32 tiles actually caused the sparsity gain to collapse because a large window is more likely to contain at least one non-zero value, preventing the row from being stripped.

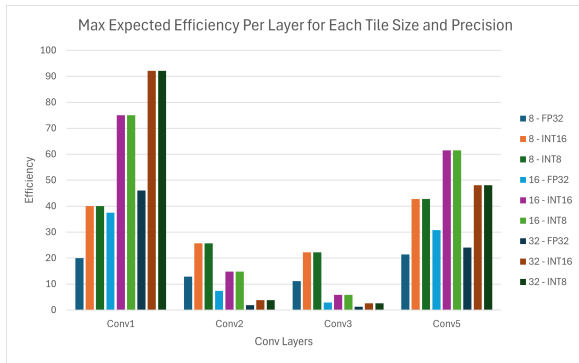


Figure 8: Theoretical Maximum Efficiency Achieved for each Tile and precision flexibility

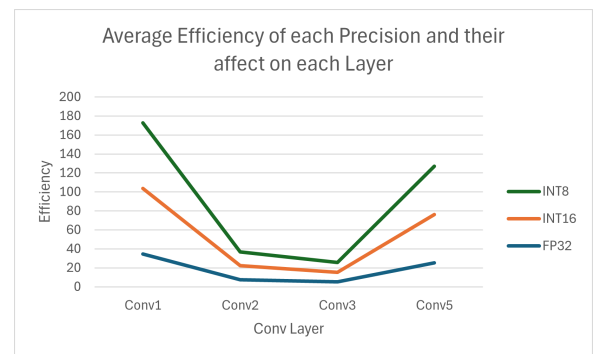


Figure 9: Theoretical Average efficiency achieved for each Quantisation precision

The most significant finding from this testing phase was the inverse relationship between tile size and sparsity gains in dense layers. It was evident more than ever that a static hardware design would be fundamentally inefficient, and the data provided the definitive proof. Despite my attempts with DPR, I realised that even if I could have passed the compilation stage, I would not have been efficient with it. The NPU did not require a physical change in its shape or anything; it simply needed a Multi-Mode Control Unit that could achieve high-latency reduction across all layers without introducing routing instabilities or control overhead from DPR. This realisation allowed me to design the architecture that would soon to be the final implementation of this project before my term ended.

3.4 Phase 3: Multi-Core NPU

The initial stage of the current implementation research involved moving away from static memory initialisation towards a proper live inference method. Since I did not have a functional Linux bootloader for Arria 10 HPS, I pivoted to using the JTAG-to-Avalon Master Bridge. This allowed the computer to act as the system host, but it introduced significant bandwidth and synchronisation challenges.

I quickly realised the massive disparity between the NPU's processing speed and the JTAG communication rate. While the NPU could compute the 32x32 matrix in approximately 700 ns, the JTAG bit-serial protocol required nearly 50ms for the same data transfer. Despite being aware of the bottleneck JTAG would produce, I did not expect such a difference when analysing it first-hand. I had to architect the system around this constraint, treating the JTAG link as more of a verification bridge for runtime reconfigurability.

To manage the FPGA registers, I developed a suite of TCL scripts. This involved mastering a specific set of Intel service commands: `get_service_paths` to identify the bridge and `master_write_32` to map data into the hardware buffers. I established a memory map for the Data, Weight, output buffers, and the NPU control registers, which made reading and writing much easier.

This was quite a valuable technical learning for me, since I had not previously learnt or heard of a TCL script. And hearing from the industry that it is a commonly required skill for FPGA engineers motivated me to learn how to operate this, so I can confidently include it in my skill set for future FPGA projects or FPGA engineering roles.

During this stage of the TCL scripting, I encountered a major hurdle when the `master_write_32` command began freezing the system for 60 seconds before crashing with a null pointer error. After a deep dive into the platform designer, I discovered a Master Reset disconnection; the master reset was not correctly driving the memory and NPU reset lines, causing the Avalon bus to hang indefinitely during a write request.

Initially, I attempted to link the Python script directly to the System Console via Java subprocesses, but an "UnsatisfiedLinkError" prevented Python from finding the Intel JTAG drivers. Without wasting further time since I had already restarted the project before, I eventually made a File-Based Handshake connection as a compromise. I turned the TCL System Console into a persistent listener that monitored a local directory for a Python-generated binary file containing data for each tile.

This introduced a new set of Race Conditions. I noticed the hardware was returning zeros, and my guess at the time was that the TCL script was somehow faster than the disk drive; it would start reading the binary file while Python was halfway through writing it. For this I tried an extra state in my VHDL wrapper file in my VHDL FSM to hold the final Avalon address stable for an extra clock cycle, ensuring the slave had enough holding time to latch the final word of the matrix.

Whilst I was struggling with the JTAG interface, I began developing an OpenVINO-based software stack to replace my previous, more limited PyTorch implementation. I switched to OpenVINO because it operates on an Intermediate Representation (IR), which consists of a .xml file containing the graph and a .bin file containing the weights. This allowed me to tell the OpenVINO engine exactly which layer data I wanted to extract without waiting for a full forward pass. I developed a layer extraction function that pulls FP32 tensors, finds the maximum value to calculate a scaling factor, and casts them to INT16 or INT8 specifically for the NPU. A critical discovery during the phase was the necessity of Hardware-Software Memory Alignment. Because the systolic array requires a fixed 32x32 grid to maintain its internal indexing, I had to update my Python function to pad the tiles into a 32x32 NumPy array; otherwise, the calculation would throw an error, which I found later to be a silly mistake on my end.

By mid-February, I noticed persistent errors, and at times, I could not tell whether I had made a logical error or the TCL JTAG system was inadequate for passing data like that. So I hypothesised that the

JTAG latency was causing the FIFO to underflow. This was my primary driver for my next major move, which involved using the Nios V processor as a local controller to eliminate race conditions.

I initiated the transition to a Nios General Purpose Processor to act as the NPU's local master. This process immediately exposed a frustrating fragmentation in Intel's software ecosystem. While the hardware was synthesised in Quartus Pro 24.1, the Nios V build tools, and Ashling RiscFree IDE were exclusively bundled with the Quartus Standard Edition. This forced me into a split development environment, where I had to generate the Board Support Package (BSP) files via the Standard Edition command line. I spent a couple of days troubleshooting CMake build errors and manually generating the BSP because Nios V lacked the legacy GUI wizards that were previously available in the software I had used throughout my time in university, which established a more manual workflow for loading the generated ELF via Ashling for hardware debugging.

To bridge my Python environment with the Nios V, I implemented the JTAG-UART IP. The goal was to use the intel-jtag-uart Python API to blast raw byte streams into the FPGA's internal FIFO, rather than waiting for a JTAG success message for each 32-bit word. On the hardware side, I wrote C drivers for the Nios V that utilised volatile pointers to read these incoming bytes and write them directly to the NPU's mapped memory addresses.

During the stress testing with larger matrices, the Nios V implementation revealed a critical FIFO overflow. Because the Python script was streaming data faster than the Nios V could process the UART interrupts and commit the data to BRAM, the internal 64-byte FIFO would saturate, leading to data corruption or no data being displayed.

I realised that the Nios V approach in this case was actually more prone to the race conditions than the host-based TCL master. And so, I decided to abandon the Nios V detour and return to the TCL-based stack.

By late February, I refocused my efforts on the core NPU hardware to address a drifting issue I observed in large matrices. I discovered that because the data and weight matrices have different dimensions, $M \times K$ and $K \times N$, using a single shared counter was quite a bad idea, causing the weight matrix to wrap prematurely. To fix this, I split the FSM into two sequential phases that involved fetching data and weight matrices separately, using separate counters.

However, during my switchback from the NIOS system, my platform designer topology, which includes the NPU core and other components, reset itself. So, the addressing of the Avalon-MM is in words, not symbols (bytes), which had been causing aliasing errors that I was trying to debug for at least three to four days. After switching that back and also ensuring that Python and TCL sent compact tiling without padding, I finally achieved 100% bit-accuracy between the quantised AlexNet model and the hardware output. With the dense baseline verified, I moved to implement a multi-precision capability. I instantiated two systolic arrays side by side: a 16-bit DSP-based core and an 8-bit LUT-based core. I developed a configuration register in the top-level wrapper to act as a hardware multiplexer; when Python sends a mode flag in the tile header, the hardware instantly routes data to the correct engine without requiring a recompile

Finally, I addressed a systematic accumulation bug that caused hardware results to be added to previous tiles. I found that the PE enable signal remained high after the tile finished, preventing the accumulator from clearing. I modified the control unit to force the internal enable signals low at tile completion as a full reset, allowing the accumulators to reset before the next tile started.

At this point, I had successfully verified a dual-core system, so my primary focus shifted to achieving a clock frequency of 100 MHz without timing violations, as discussed in Section 3.4.1. On the side, I considered moving to a 3-core design by adding a third Int16 16x16 array alongside the existing architectures, as a sort of middle ground. However, this experiment quickly became a major technical hurdle. After recompilation, the cycle count logic broke entirely, consistently returning zeros. I also ran into persistent discrepancies in my matrix multiplications, where the entire first column was

completely incorrect. This went on for several days, and I tried various methods to debug, including running QuestaSim simulations. This is when I noticed that I had somehow lost the software license, and it would take IT a few days to come fix it. I realised I had no choice but to wait 20-50 minutes at a time for a compilation after trying something to fix the issue. After all the failures, including a particularly frustrating attempt that resulted in all-zero outputs because I thought it might be a logic-congestion issue that messed up the timing. So I decided to abandon the 3-core branch and revert to a cleaner, more stable 2-core architecture: a DSP-based INT16 32x32 core for high-precision tiles and a new, larger LUT-based Int8 16x16 core to replace the old 8x8 version. To fix the counter issue, I realised I was simply not latching it properly before the hardware cleared itself. I also learnt that I could add attributes to the VHDL script and reduce the fanning out that was causing the congested logic.

Once this hardware was stable, I overhauled my OpenVINO-based Python script to match the new 2-core configuration. This meant routing the tiles to either of the two systolic arrays, depending on the tile's sparsity. I also added support for structured pruning, where weight matrices contain zeros, since that's what the old PyTorch script had, and I needed a method to simulate production-level sparsity. This was perfect for showing the NPU hardware architecture at its fullest potential.

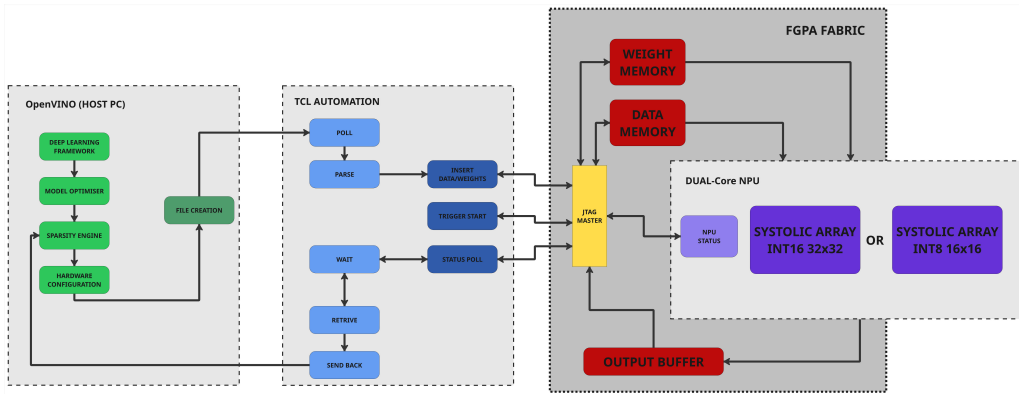


Figure 10: Complete System Diagram

A significant challenge in the current implementation was the communication bottleneck between the host PC and the Arria 10 FPGA. While the NPU core was capable of sub-microsecond inference, the JTAG-to-Avalon Master Bridge limited the system to a bit-serial level transmission. I truly noticed that during full-scale inference runs on the models I selected, I encountered several integration bugs, including file permission and timing errors, which slowed down the process quite a bit since each model took a few hours to run, given that JTAG transmission was only 24MHz, and my compromises that brought the FMAX frequency to 70MHz. As an engineer, I had to make a strategic compromise. I prioritised system stability and bit-accurate verification over raw performance.

By treating the JTAG link as a verification bridge, I demonstrated the functional correctness of the dual-core architecture across multiple vision models. This decision allowed me to deliver a fully operational, hardware-verified system within three months, while simultaneously conducting parallel research into high-bandwidth alternatives for a final result deployment.

The research conducted in the final weeks identified three possible paths for scaling NPU performance. The first involved migrating the OpenVINO host script from an external laptop to the Arria 10's HPS, which was one of the reasons I wrote the OpenVINO script in the first place. By booting a Linux kernel on the ARM Cortex-A9 cores, the openVINO inference engine could dispatch tiles directly via the high-speed bridge, eliminating external cable latency. Secondly, I evaluated the implementation of a PCIe Gen3 x16 interface. Research into the Intel Avalon-mm Arria 10 hard IP for PCIe suggested that this would provide Gbps-level throughput necessary for real-time video processing. Finally,

while investigating a 10GbE (Ethernet) network stack for remote inference, I conducted a rough risk assessment and determined that the complexity of implementing a reliable UDP/IP offload engine was too high for the remaining project timeline. Consequently, the proposed future architecture focuses on a System on Chip approach, utilising the HPS as the orchestrator.

3.4.1 Optimisations Performed

The transition from simulation to hardware synthesis was the most technically demanding phase of the project, forcing me to move beyond functional VHDL into the realm of physical timing closure. Initially, the system was capped at a 60 MHz fmax due to severe routing congestion and high logic depth within the Processing Element (PE) array. The Quartus Timing Analyser identified critical paths up to 8 logic levels deep, primarily where the control unit was attempting to mask 1,024 PEs based on dynamic tile dimensions within a single clock cycle.

To resolve this, I implemented a series of pipelining strategies. I broke these long combinational paths by inserting register stages, effectively splitting the control logic into a two-phase operation: one to latch the tile header and another to enable the array. While this added a single cycle of latency to the setup phase, it allowed the clock period to drop significantly. I also learned to utilise the max fanout and register duplication attributes in my VHDL code. By forcing Quartus to duplicate high-fanout control signals, I reduced the physical distance the global enable signal had to travel across the Arria 10 floorplan, mitigating interconnect delays that had previously caused timing failures.

This optimisation phase taught me that hardware design is a balancing act between resource usage and frequency. I spent days in the Chip Planner and Technology Map Viewer observing how my code translated into physical ALM (Adaptive Logic Module) clusters. This iterative process of synthesise, analyse, and then refactor provided a deep appreciation for the underlying silicon architecture, a level of detail rarely required in undergraduate labs but fundamental to professional academia.

3.4.2 Results

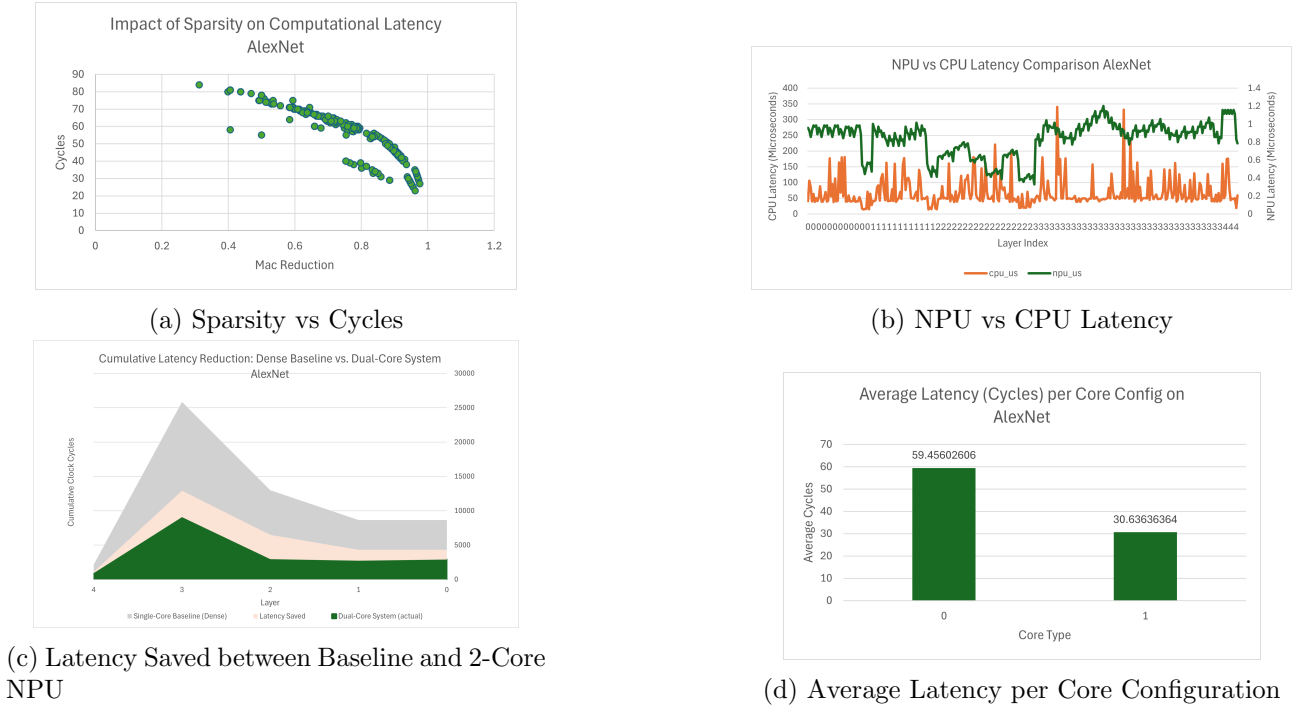


Figure 11: AlexNet Performance: 36.25% Total Latency Reduction.

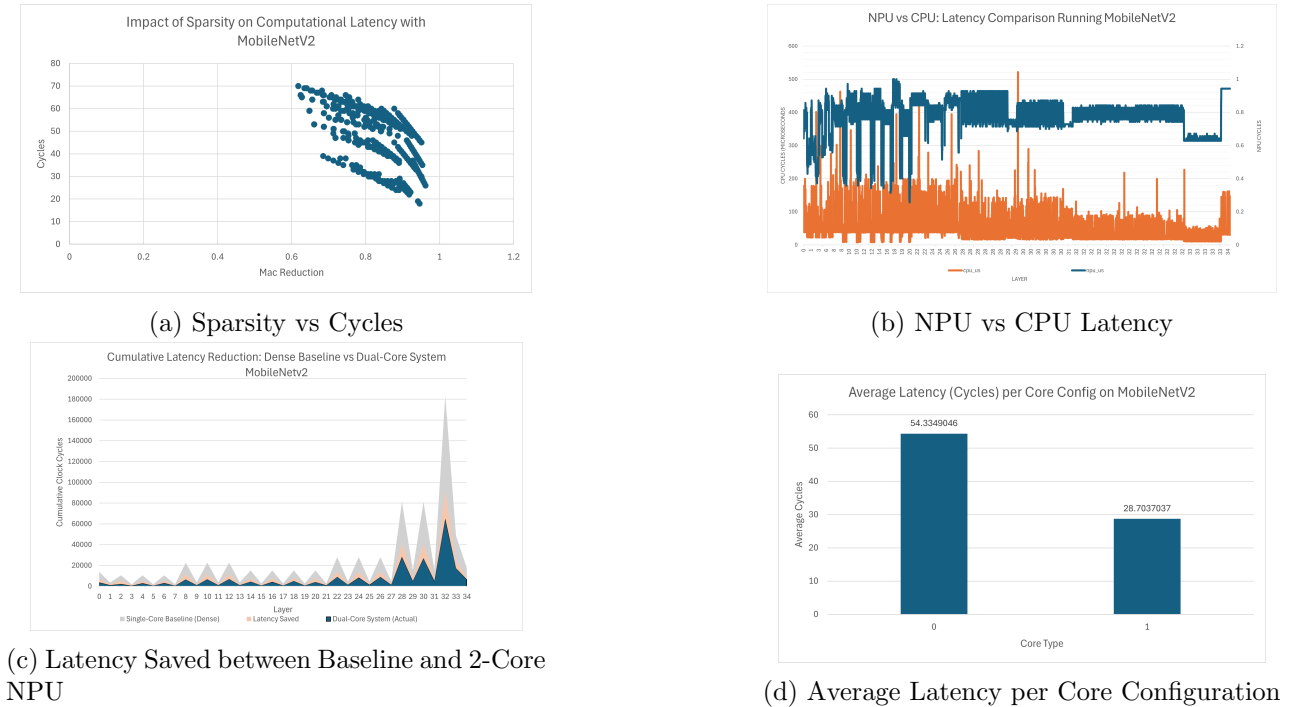
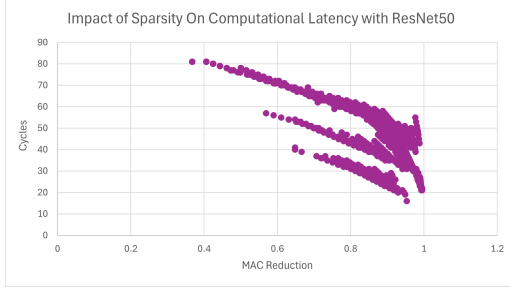
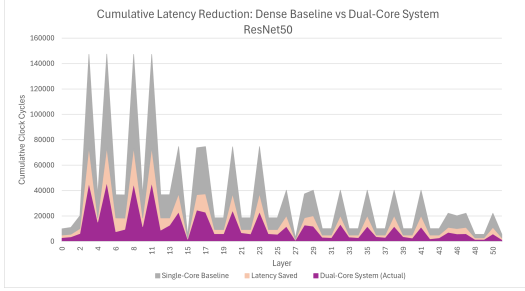


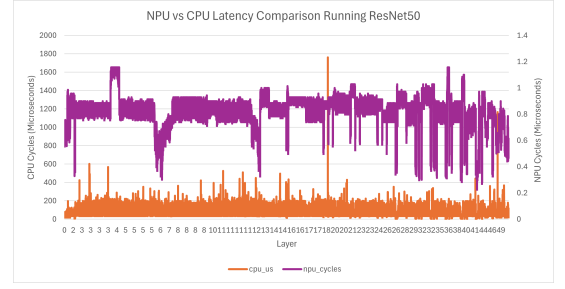
Figure 12: MobileNetV2 Performance: 34.50% Total Latency Reduction.



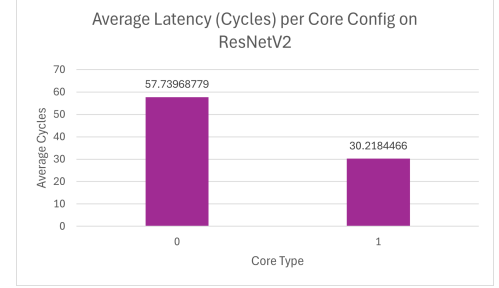
(a) Sparsity vs Cycles



(c) Latency Saved between Baseline and 2-Core NPU

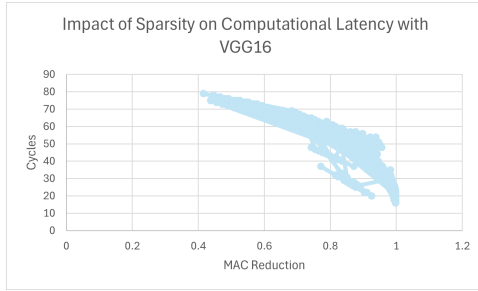


(b) NPU vs CPU Latency

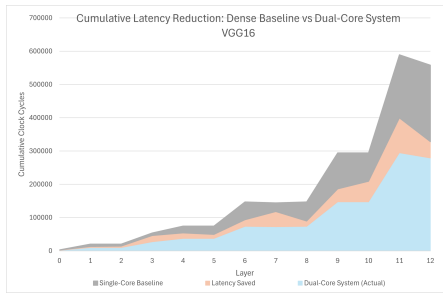


(d) Average Latency per Core Configuration

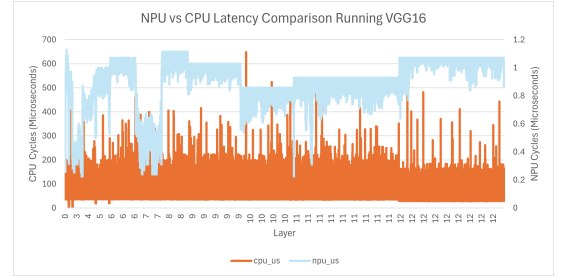
Figure 13: ResNet50 Performance: 38.16% Total Latency Reduction.



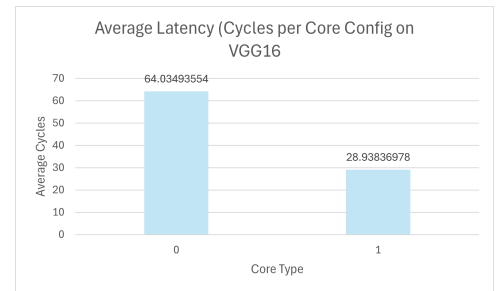
(a) Sparsity vs Cycles



(c) Latency Saved between Baseline and 2-Core NPU



(b) NPU vs CPU Latency



(d) Average Latency per Core Configuration

Figure 14: VGG16 Performance: 32.37% Total Latency Reduction.

The results of this research provide a definitive validation of the dual-core, multi-precision NPU as a high-performance alternative to static systolic architectures. By benchmarking the system across four distinct vision models, I successfully quantified a total latency reduction of 38.16% for ResNet50, 36.25% for AlexNet, 35.50% for MobileNetV2, and 32.37% for VGG16 compared to a dense baseline. These figures represent a hardware-verified proof-of-concept/prototype, in which the NPU intelli-

gently switched between a high-throughput Int16 32x32 core and an efficient Int8 16x16 core based on real-time sparsity detection. This architectural adaptability proved most effective in the spatial convolutional layers of ResNet50 and AlexNet, where the system eliminated hundreds of thousands of redundant clock cycles. And most importantly, 100% bit-accuracy was maintained across all inferences, confirming that the strategic decision to prioritise JTAG-verified stable build over high-risk experimental features successfully delivered a robust research asset to the department.

3.5 Reflective Appraisal

Throughout this research tenure, I operated within a highly project-driven environment that demanded quite a level of accountability and rigour. Unlike a structured classroom setting, my work lacked predetermined solutions, requiring me to shift from a passive student mindset to a proactive Research Engineer mindset. I quickly identified that in an open-ended R&D lifecycle, project momentum can easily stall during the hardware debugging phase if not strictly managed. To mitigate this, I implemented a personal project management framework that breaks high-level architectural goals into specific, daily micro-milestones. This structured methodology proved essential when navigating the inherent uncertainties of FPGA development. By setting internal deadlines for core deliverables, I maintained a consistent pace and executed strategic pivots when needed.

One of the most valuable lessons I learned was the true utility of a detailed logbook. Whilst I have always kept records of my work, I previously viewed them as a static requirement rather than a dynamic tool. During the symbols to words reset after I pivoted back from the Nios attempt. I spent days frustrated by errors that felt new, only to realise later that I had encountered and solved a similar issue. This was a turning point for me. I began reviewing my entries every few weeks to look back and gain a big picture perspective. I found that the logbook is not just for tracking progress, but also a repository for lessons I learnt during the work, which can prevent hours of redundant troubleshooting if consulted regularly, especially if one tends to forget as quickly as I do.

Coming into this project, I had completed approximately seven FPGA-based projects in my undergraduate studies, big or small. However, this experience taught me that there is a massive gap between making VHDL work and making it hardware-efficient. In traditional embedded systems or high-level programming, I am often shielded from the nitty-gritty timing details, so I focus on the task's operation first, then any optimisations. For FPGAs, that would essentially be a double-edged sword. If I added a fix to a logic path, that could actually create new timing violations elsewhere. So I learnt how to navigate the entire Quartus analysis system to see how the logic depth, delays, and resources are affected, which at times has been much more important than the hardware code itself.

I learned the importance of scalability and knowledge transfer within a research lifecycle. In a university R&D environment, a project does not end with the individual; it serves as a foundation for future PhD/Master's candidates. Recognising this, I ensured my focus was not only on achieving functional hardware but also on curating a comprehensive Project Compendium. This included a structured research report detailing all the technicalities of the current implementation. This ensures that any future researchers who continue this project do not encounter the same issues I did. For this, I had to ensure that my logbook served as a comprehensive repository of system-level discoveries, since my GitHub repository was not allowed to be publicly released. This documentation was enhanced over time, which included roadmaps, optimisation methods, failed attempts, etc. The commitment to documentation, whether in the logbook or in cohesive, well-commented code, had to ensure a project could be resumed at any given point, and that is vital in professional engineering. It differentiates a temporary undergraduate project from a lasting contribution to the research department's Intellectual Property.

This experience reinforced the importance of critically evaluating complex design approaches and making high-level decisions on when to strategically pivot. Operating with biweekly supervisory

input focused primarily on high-level research objectives meant that I was responsible for the entire technical execution and local problem-solving. Rather than viewing the absence of daily technical support as a hurdle, I treated it as an opportunity to exercise professional engineering judgment. I was responsible for independently diagnosing system-level conflicts, often through deep dives into Intel Arria 10 datasheets and timing reports to identify where theoretical assumptions diverged from actual hardware tendencies. This taught me to 'own' the technical roadmap; I learned that the willingness to objectively abandon a dead-end branch of research in favour of a robust, alternative implementation is a good representation of technical maturity. These skills of independent diagnostic thinking and decisive project pivoting are core competencies that I will carry forward into my career as a Computer Systems Engineer.

Beyond the technical challenges, what was rewarding in this R&D role was the creative freedom to architect a system from scratch. Unlike a structured classroom lab where the solution is predetermined, this experience gave me a glimpse of what it might be like in an R&D department in industry. I particularly enjoyed the investigative nature of the work, as a lot of this research is fairly new. There was a genuine sense of excitement in uncovering the differences within a vision model like AlexNet and then engineering a Multi-Mode system to exploit it. This project truly helped me shift my perspective from a series of calculations to a form of creative problem-solving that I did not encounter as much in university group projects. It confirmed that I am most engaged when operating at intersections of large systems, where I have the ability to research, fail, and eventually build a foundation that pushes the boundaries of the department's current capabilities

3.6 Conclusion

My time as a Summer Research Assistant has been a transformative experience that bridged the gap between my undergraduate studies and the demands of professional engineering R&D. By successfully developing and verifying a dual-core NPU that achieved 30-40% latency reduction, I have demonstrated the practical value of exploiting sparsity in hardware.

The work has helped me develop a wide range of technical skills, from VHDL optimisations and TCL scripting to high-level model characterisation via OpenVINO. Beyond the technical output, I have gained critical professional soft skills, including the ability to manage a high-stakes project with ease, diagnose complex system-level conflicts, and pivot strategically when faced with technical dead ends, as in the initial DPR phase.

This experience has solidified my interest in the intersection of AI hardware and embedded systems. By delivering a comprehensive Project Compendium and an accurate hardware prototype, I have contributed a lasting asset to the department's research capabilities. I am confident that the technical resilience and independent problem-solving skills I have honed during these few months will be invaluable as I move forward in my career.